

LOGGING

Il **logging** è il processo di registrazione di messaggio eventi importanti generati da un'applicazione software durante la sua esecuzione.

Questi messaggi di log possono includere informazioni su errori, avvisi, eventi significativi, operazioni completate e altri dati rilevanti.

Il logging è essenziale per monitorare il comportamento di un'applicazione, diagnosticare problemi e raccogliere informazioni utili per il debugging e l'analisi delle prestazioni.

I messaggi di log vengono inseriti nel codice dal programmatore, il quale decide dove inserirli in funzione di ciò che ritiene importante monitorare o diagnosticare.

Livelli di logging:

- **DEBUG:** Messaggi di log per il debugging del codice. Il livello debug si divide in **fine**, **finer**, **finest**.
- **INFO:** messaggi di log per reperire informazioni sulla normale esecuzione del programma.
- **WARNING:** avvisi su situazioni potenzialmente problematiche che non interrompono l'esecuzione del programma.
- **ERROR:** errori che possono influenzare il funzionamento dell'applicazione, ma non provocano il blocco totale del programma.
- **FATAL:** errori critici che causano l'interruzione dell'esecuzione del programma.
- **SEVERE:** errori gravi che potrebbero causare il fallimento dell'applicazione.

Fine: è il livello più generico tra i tre. Si usa per registrare eventi importanti ma non troppo dettagliati. Ad esempio, il completamento di un'operazione o l'inizio di un processo:

- Esempio: *"L'inizializzazione del modulo è stata completata."*

Finer: offre un livello di dettaglio maggiore rispetto a fine. È ideale per catturare passaggi più specifici o intermedi di un'operazione.

- Esempio: *"Il file di configurazione è stato trovato e sta per essere caricato."*

Finest: questo livello registra ogni singolo dettaglio possibile per avere una traccia estremamente approfondita del funzionamento interno.

- Esempio: *"valore del parametro 'X' = 42, ciclo iterazione = 3."*

La classe Logger in Java fa parte del pacchetto `java.util.logging` ed è uno strumento fondamentale per registrare messaggi di log durante l'esecuzione di un'applicazione.

I logger vengono creati utilizzando il metodo statico

```
Logger.getLogger(String name)
```

dove `name` è un nome identificativo per il logger (solitamente si assegna al logger il nome della classe in cui viene creato).

Il logger utilizza i gestori (Handler) per definire dove salvare i messaggi (esempio: console, file, ecc.). L'Handler in Java è una classe astratta fornita dalla libreria `java.util.logging`.

Un Handler in Java rappresenta un'entità responsabile di **gestire e inviare i messaggi di log a un determinato output**, come un file, la console, o un server remoto. Gli Handler lavorano in combinazione con il Logger e i suoi livelli di log.

Sottoclassi della classe Handler:

- **ConsoleHandler:** invia i messaggi di log alla console (standard output o terminale).
- **FileHandler:** scrive i messaggi di log su un file.
- **SocketHandler:** invia i log a un server remoto tramite socket.

In Java l'Handler di default è `ConsoleHandler`.

Utilizza anche `Formatter` per formattare i messaggi.

`Formatter` è una classe astratta che fa parte del pacchetto `java.util` ed è utilizzata per creare stringhe formattate o, nel caso del logging, per personalizzare l'aspetto dei messaggi di log. Con la classe `Formatter` puoi specificare il layout, lo stile e il contenuto dei messaggi.

Esempio:

```
[2025-03-11 11:54:15] [INFO] Questo è un messaggio formattato.
```

`SimpleFormatter` è una classe concreta in Java che estende la classe astratta `Formatter` ed è fornita di default nel pacchetto `java.util.logging`. Come suggerisce il nome, `SimpleFormatter` è una soluzione semplice per formattare i messaggi di log con un formato predefinito.

Quando si utilizza `SimpleFormatter`, il messaggio di log viene formattato automaticamente con il seguente schema:

```
mar 11, 2025 11:54:15 AM SimpleFormatterExample main
```

```
INFO: Questo è un messaggio INFO.
```

```
mar 11, 2025 11:54:15 AM SimpleFormatterExample main
```

```
WARNING: Questo è un messaggio WARNING.
```

Nota bene: `main` indica il nome del **metodo** che ha prodotto il messaggio di log.

```

import java.util.logging.ConsoleHandler; //puo' non essere incluso
import java.util.logging.Logger;
import java.util.logging.SimpleFormatter; //puo' non essere incluso
import java.util.logging.Level;

public class EsempioLogging {
private static final Logger logger=Logger.getLogger(EsempioLogging.class.getName());

    public static void main(String[] args) {
        // Creazione di un handler per la console
        ConsoleHandler consoleHandler = new ConsoleHandler();

        // Impostazione di un formatter semplice per i messaggi di log
        consoleHandler.setFormatter(new SimpleFormatter());

        // Aggiunta dell'handler al logger
        logger.addHandler(consoleHandler);

        // Disabilitazione degli handler predefiniti
        logger.setUseParentHandlers(false);

        // Impostazione del livello di log
        logger.setLevel(Level.ALL);
        consoleHandler.setLevel(Level.ALL);

        // Log di messaggi di vari livelli
        logger.info("Messaggio informativo");
        logger.warning("Messaggio di avvertimento");
        logger.severe("Messaggio di errore grave");
    }
}

```

Nota bene:

setFormatter() è un metodo della classe astratta Handler che fornisce le funzionalità base per impostare un oggetto di tipo SimpleFormatter.

```

import java.util.logging.Logger;
import java.util.logging.Level;

public class EsempioLogging {
    private static final Logger logger=Logger.getLogger(EsempioLogging.class.getName());

    public static void main(String[] args) {
        // Messaggi di log a diversi livelli
        logger.setLevel(Level.ALL);
        // Impostiamo il livello di logging a ALL per mostrare tutti i messaggi
        logger.log(Level.FINE, "Messaggio di debug - dettagli");
        logger.info("Avvio dell'applicazione");

        try {
            // Codice che potrebbe generare un'eccezione
            int risultato = 10 / 0;
            System.out.println("Il risultato è: " + risultato);
        } catch (Exception e) {
            logger.log(Level.SEVERE, "Errore durante il calcolo: ", e);
        }

        if (someCondition()) {
            logger.warning("Attenzione: condizione anomala rilevata");
        }

        if (criticalCondition()) {
            logger.log(Level.SEVERE, "Errore critico: interruzione dell'applicazione");
            // Eventualmente terminare l'applicazione
        }

        logger.info("Fine dell'applicazione");
    }

    private static boolean someCondition() {
        // Implementazione di una condizione
        return true;
    }

    private static boolean criticalCondition() {
        // Implementazione di una condizione critica
        return false;
    }
}

```

Di default, infatti, Java configura automaticamente un gestore di tipo `ConsoleHandler` con il suo formatter (`SimpleFormatter`), quindi non è necessario configurarli manualmente per visualizzare messaggi sulla console.

Il `ConsoleHandler` è già incluso e configurato di default nel sistema di logging `java.util.logging`.

Per molti casi comuni il sistema di logging di Java si occupa di tutto automaticamente, ma i componenti `ConsoleHandler` e `SimpleFormatter` non sono superflui, anzi possono essere strumenti utili per scenari più avanzati o personalizzati.

OSSERVAZIONI GENERALI

- `logger.setLevel()`: metodo che imposta il livello di log per il logger.
- `consoleHandler.setLevel()`: metodo che imposta il livello di log per il `ConsoleHandler`.
- `info()`, `warning()`, `severe()`: sono metodi appartenenti alla classe `Logger`.
- `logger.setUseParentHandlers(false)`: per disattivare l'handler predefinito.

DIFFERENZA TRA LOGGING ED ECCEZIONE

In Java, **logging** ed **eccezioni** sono strumenti diversi, ma complementari, utilizzati per gestire e monitorare il comportamento di un'applicazione.

Logging

Definizione: il logging è il processo di registrazione di informazioni rilevanti durante l'esecuzione di un programma. Queste informazioni possono includere messaggi sullo stato dell'applicazione, avvisi, errori, o semplicemente dati utili per il debug.

Scopo: è principalmente usato per monitorare il funzionamento del sistema, diagnosticare problemi o tracciare eventi importanti.

Utilità: i log sono spesso salvati in file o console e aiutano i programmatori ad analizzare il comportamento dell'applicazione.

Eccezioni

Definizione: Le eccezioni rappresentano situazioni anomale o errori che si verificano durante l'esecuzione di un programma, come un accesso a un array fuori dai limiti o un file non trovato.

Scopo: Servono per gestire errori in modo strutturato, garantendo che il programma possa reagire adeguatamente senza bloccarsi in modo improvviso.

Strumenti in Java: Java utilizza classi come `Exception` e `RuntimeException` per rappresentare le eccezioni.

Differenza principale

Logging: utilizzato per registrare informazioni sullo stato del sistema, indipendentemente dal fatto che si verifichino errori o meno. È un modo per scrivere monitorare ciò che accade, sia normale che insolito, così da poterlo analizzare in seguito.

Eccezioni: meccanismi specifici per gestire errori o eventi inaspettati durante l'esecuzione. Sono usate per gestire **problemi specifici** mentre il programma è in esecuzione.